AETHER

Backend and Agents

The backend stack, the AWS services by the moment each one acts, and each agent's tools and routing conditions.

Project Autonomous Margin & Collateral Intelligence **Programme** GIFT IFIH Young Builders **Revision** B
Generated 2026-08-22

> **PROTOTYPE DOCUMENT**
> Describes a prototype under construction. All market data, counterparties, portfolios and legal documents referenced are **simulated** and clearly labelled as such in the repository. Nothing here asserts regulatory approval, certification under any industry methodology, or connectivity to any market infrastructure.

All data described here is **simulated**. No transaction is executed, no market infrastructure is connected, and no regulatory approval is claimed.

Part 1 — The backend

The rule the stack is arranged around

> Language models extract. Deterministic engines compute.

Every library below sits on one side of that line, and the arrangement is what makes the claim structural rather than procedural.

Runtime

TOOL	VERSION	WHERE IT SITS	WHY THIS ONE
Python	3.12+ (3.13 on Lambda)	everywhere	`Decimal`, `StrEnum`, structural pattern matching in the tool dispatcher
FastAPI	0.141.1	`api/`	Thin routers only. A route parses, calls one engine or agent function, serialises, returns
Starlette	1.6.0	under FastAPI	ASGI, and the SSE `StreamingResponse` behind run events
Pydantic	2.13.4	`agents/schema.py`	The typed boundary a model's output must pass to become engine input
pydantic-settings	2.15.0	`config.py`	`AETHER_MODE`, `AETHER_ENGINE_VERSION` and friends, resolved once
NumPy	2.5.2	`core/sensitivities.py`	Risk-weight and correlation matrices
SciPy	1.18.1	`core/optimize.py`	`linprog` over HiGHS — the collateral LP, and its duals
pypdf	6.16.1	`agents/legal_clause.py`	Native PDF text layer. A scanned image has none and is refused, not guessed at
Mangum	0.21.0	`deploy/lambda_handler.py`	ASGI to Lambda. The FastAPI app is unchanged
Uvicorn	—	local only	Development server. Not in the deployment package

TOOL	VERSION	WHERE IT SITS	WHY THIS ONE
Anthropic SDK	—	lazy import	`live` mode only. Not in the deployment package

Declared in `pyproject.toml` but never imported: `sqlmodel` (drags in SQLAlchemy), `python-multipart` (no upload endpoint exists), `certifi`. `truststore` is imported lazily and only when `AETHER_USE_TRUSTSTORE` is set, for the TLS interception on the build machine.

Testing and quality

TOOL	PURPOSE
pytest	487 tests, marked `golden`, `property`, `determinism`, `slice`
Hypothesis	property tests — invariants over generated inputs, not chosen examples
httpx	API-level tests through the ASGI app
Ruff	lint and format
mypy	type checking across `backend/aether`

Golden expected values are derived **analytically, before the implementation**: a flat curve discounts at `exp(-rt)`; a swap struck at its own par rate is worth zero. A golden test whose expected value came from running the code proves only that the code is deterministic.

`tests/unit/test_core_purity.py` parses every module under `core/` with the `ast` module and fails the build on a forbidden import, an implicit clock, or module-level mutable state. The deterministic-core rule is enforced by CI, not by discipline.

Why `core/` has no dependencies

`backend/aether/core/` — 18 modules — imports no framework, opens no socket, touches no database and never calls `datetime.now()`. Every function maps values to values.

That is what makes two things possible at once: golden tests with hand-derived expected values, and audit hashes that are byte-identical across runs. `Decimal` at every monetary boundary, never `float`; floats are permitted inside curve interpolation and LP solving, but anything that becomes a cash amount is `Decimal`. Money crosses the API as a **decimal string**, never a JSON number, because a nine-figure notional loses cents through a JavaScript double.

Part 2 — AWS, classified by when it acts

Region `ap-south-1`. No containers, no VPC, no CloudFormation or Terraform — deployment is a bash script plus a hand-wired distribution.

At build time — the developer's machine

SERVICE	USED FOR	WHEN
STS	`get-caller-identity` resolves the account number that names the role and bucket	Every `deploy.sh` invocation
S3 (`aether-deploy-…`)	Staging a package too large for `--zip-file fileb://`	Only above the 50 MB direct-upload limit

At deploy time — `deploy.sh`

SERVICE	USED FOR	WHEN
IAM	Creates `aether-lambda-role`; attaches `AWSLambdaBasicExecutionRole`	Created if absent, adopted if present
Lambda	`update-function-code`, `update-function-configuration`, `put-function-concurrency`	Every `deploy.sh backend`
S3 (`aether-site-…`)	`s3 sync` of the Next.js static export, public access fully blocked	Every `deploy.sh frontend`
CloudFront	*Not scripted.* The distribution and both origins are wired by hand	Once, manually

At request time — serving a browser

SERVICE	ROLE IN ONE REQUEST
CloudFront `E2XYCWF9L9C796`	Single entry point. Default behaviour → S3; `/api/*` → API Gateway. One domain for both is what makes every browser request same-origin, so there is no CORS and no preflight
CloudFront OAC `aether-s3-oac`	Signs CloudFront→S3 with SigV4, so the bucket stays entirely private
S3 `aether-site-…`	Serves the static export. `trailingSlash: true`, so `/review/` resolves and `/review` may not
API Gateway (HTTP API `gwrvbykv32`)	The real `/api/*` origin. Cache policy `CachingDisabled`, origin request policy `AllViewerExceptHostHeader` — forwarding the viewer `Host` makes API Gateway reject the request
Lambda `aether-api`	python3.13, 2048 MB, 30 s timeout, reserved concurrency 20. Mangum adapts the event to ASGI; FastAPI is unchanged

At observation time — diagnosis

SERVICE	WHAT IT ANSWERS
CloudWatch Logs `/aws/lambda/aether-api`	Import failures, tracebacks, `INIT_REPORT` and per-request duration
CloudWatch Metrics	`Invocations`, `Errors`, and critically `Throttles`

A throttled invocation writes no log line. It never reaches the application, so the only evidence is the `Throttles` metric. A reservation of 1 once starved the dashboard's parallel opening fetches and surfaced in the browser as a bare `503` on an API that was, request by request, entirely healthy.

Provisioned but unused

Named here so they are not mistaken for architecture.

- **Lambda Function URL** — exists with `AWS_IAM` auth, but the distribution routes to API Gateway instead.
- `aether-lambda-oac` — created; the API origin carries no OAC.
- **DynamoDB** — **no table exists**, and no `boto3` import appears anywhere in `backend/aether/`. The IAM policy granting three actions on table `catherine` grants them on nothing. Review-workflow state lives in process memory, and `api/main.py` says so.

Part 3 — The three agents

Three agents, each owning one stage, each handing a typed result to the next.

```
Legal-Clause Agent      reads the Annex, cites, flags        -> CSATerms
        |
        v   frozen, reviewed terms
Risk-Engine Agent       routes valuation and margin tools    -> exposure, margin call
        |
        v   credit support amount
Collateral-Optimizer    routes optimisation and settlement   -> allocation, instruction
        |
        v
Decision (proposal only, pending human approval)
```

The boundary all three share

An agent chooses **which tool to call and with what references**. The tool computes. That is not a convention — it is the type of a tool call's arguments:

```
Argument = str | int | bool | None
```

A `ToolCall` whose arguments contain a `float`, a `Decimal` or a `Money` raises `TypeError` at construction. An agent therefore cannot pass a number that becomes a cash amount, even if a model hallucinated one into the plan. Monetary values exist only inside typed engine results, which flow between tools through the run context and never through a plan.

Every tool call produces two chained audit events:

1. `TOOL_CALLED` — actor type `AGENT`, carrying tool, intent and arguments. It contains no monetary value, because it cannot.
2. the engine's `*_COMPUTED` event — actor type `ENGINE`, carrying the numbers.

A reader of the chain can verify, event by event, that no figure originated on the agent side.

`TOOL_REGISTRY` is the only door to the engine — six tools, each naming its owning agent, its required arguments and what it produces. `validate_plan()` rejects a plan that is empty, names an unknown tool, assigns a tool to the wrong agent, or omits a required argument.

Agent 1 — Legal-Clause

Input a CSA, markdown or native PDF. **Output** an `ExtractionResult`: typed, cited, confidence-scored fields.

It never computes a margin number, never invents a value absent from the text, and never emits a field without a citation — *a field without a citation is an assertion, not an extraction.*

Two modes, one output type. `replay` reads a cached extraction keyed by the document's SHA-256: no network, no key, byte-identical. This is the demo path. `live` calls the model at `temperature=0` with a pinned model id and a versioned prompt, then validates the response against the same schema. Nothing downstream can tell which ran, beyond the `mode` field recorded for the audit trail.

WORKFLOW CONDITIONS — WHEN A FIELD STOPS THE PIPELINE

A field is routed to a human when `requires_review` is set by the agent **or** its confidence falls below `CONFIDENCE_THRESHOLD = 0.85`. `ReviewReason` records which:

REASON	MEANING
`LOW_CONFIDENCE`	Below the threshold

REASON	MEANING
`CONDITIONAL_CLAUSE`	The value depends on a state the document defines but cannot resolve
`AMBIGUOUS`	The text supports more than one reading
`NOT_FOUND`	The field is absent from the document
`CONFLICTING_CLAUSES`	Two paragraphs disagree

`CONDITIONAL_CLAUSE` is the interesting one. Paragraph 11(b)(iii)(A) sets Party B's Threshold at USD 5,000,000 — but "subject to" another paragraph that drops it to zero while a ratings downgrade is in effect. The document defines the trigger. It cannot say whether the trigger is currently pulled. That is not a question a document can answer, so the agent scores it 0.52 and stops.

THE GATE IS STRUCTURAL

`freeze_terms()` is the only path from `ExtractionResult` to `CSATerms`, and it raises `ReviewRequired` while any field awaits a decision. The API returns `409` rather than computing.

```
extraction = extract_terms("CSA-MERIDIAN-2026")
freeze_terms(extraction, ...)          # ReviewRequired: threshold.party_b
confirmed = confirm_field(extraction, "threshold.party_b", reviewer, rationale)
terms = freeze_terms(confirmed, ...)   # now permitted
```

`confirm_field()` refuses a decision with no rationale — *decisions must be explainable*. Every field carries a `FieldSource` of `AGENT` or `HUMAN`, and the two stay distinguishable forever.

AUDIT EVENTS

EVENT	ACTOR TYPE	RECORDS
`DOCUMENT_INGESTED`	`SYSTEM`	The content hash the whole run is anchored to
`TERMS_EXTRACTED`	`AGENT`	Model id, prompt version, mode, temperature, field and citation counts, mean confidence, the flagged paths
`FIELD_CONFIRMED`	`HUMAN`	One per reviewer decision, with the rationale
`TERMS_FROZEN`	`HUMAN`	The version the engine is permitted to compute on

Without these the chain proves only that arithmetic happened. With them it proves where every contractual value came from and who accepted it.

Agent 2 — Risk-Engine

Input frozen `CSATerms`, a portfolio, a pinned `MarketSnapshot`, an `as_of` date. **Output** exposure, sensitivity margin, schedule initial margin, and the margin call.

By construction its terms can only have come from `freeze_terms()` after every flagged field was resolved.

ITS FOUR TOOLS

TOOL	REQUIRED ARGUMENTS	PRODUCES
`price_netting_set`	`netting_set_id`, `snapshot_id`	Exposure — marks every trade and nets **within that set only**

TOOL	REQUIRED ARGUMENTS	PRODUCES
`measure_risk_sensitivities`	`netting_set_id`, `snapshot_id`	Bucketed delta by full revaluation, risk-weighted, correlated
`compute_schedule_initial_margin`	`netting_set_id`, `snapshot_id`	BCBS-IOSCO standardised schedule
`compute_credit_support_amount`	`agreement_id`	The CSA waterfall

Every argument is an identifier. Not one is a number that could become money.

WORKFLOW CONDITIONS

Valuation and the credit support waterfall are **always** required. `measure_risk_sensitivities` is **skipped for an empty book** — there is nothing to perturb. If the sensitivity result is still absent when the swarm completes, it is computed directly rather than left null.

Cross-Master netting is structurally prevented, not merely unimplemented.

THE CALCULATION THIS PRODUCT EXISTS TO GET RIGHT

```
CreditSupportAmount = max(0, IA + max(0, Exposure) − Threshold)
DeliveryAmount      = CreditSupportAmount − PostedValue
gate                |Amount| < MTA  ->  no transfer
round               ROUND_UP for delivery, ROUND_DOWN for return
```

Whether the MTA is tested before or after rounding is genuinely ambiguous across CSA vintages and is a real source of collateral disputes. Both conventions are supported, the agreement elects one, and **the trace records which was applied**.

Neither initial margin figure is ISDA SIMM. One is the public BCBS-IOSCO schedule; the other is a risk-weighted sensitivity aggregation using AETHER's own documented weights, calibrated to nothing and certified by no one.

Agent 3 — Collateral-Optimizer

Input the margin call, the eligible inventory, the pinned snapshot. **Output** an allocation and a settlement instruction — or, deliberately, neither.

WORKFLOW CONDITIONS — GENUINELY CONDITIONAL, ON FOUR GROUNDS

This is the agent whose routing actually branches. `collateral_optimizer_plan()` decides:

CONDITION	DECISION	WHY
No call (`has_call` false)	No tool invoked at all. A `NO_TOOL_REQUIRED` event is written	The Threshold and MTA gates gated. There is nothing to select
Return Amount	Settlement instruction only, no LP	Collateral is released from the balance already held, so there is nothing to choose
Delivery Amount owed by the counterparty	Settlement instruction only, tagged with the transferor	Optimising *our* inventory against *their* obligation would be nonsense. The correct output is a margin call request; their proposal comes back in reply
Delivery Amount we owe	`optimise_collateral`, then `generate_settlement_instruction`	The only path that reaches the linear program

An infeasible allocation escalates rather than proceeding: the optimiser's status is carried into the outcome, and a non-`OPTIMAL` result returns its message instead of a cost.

THE RANKING THAT DECIDES THE ALLOCATION

```
cost_per_effective_unit = funding_spread_bps / effective_valuation_percentage
```

Indian government paper at a 9 bp funding spread and an 86.48% effective valuation (94% schedule × 92% FX) costs 10.41 bp per unit of collateral actually delivered. A desk ranking by headline spread would reach for it ahead of a 98%-valued Treasury and be wrong.

On the committed fixture the optimiser **evaluates the rupee paper and declines it**, because the Treasury bill covers the residual at 8.04 bp. That is the honest result and the screen reports it with the reason attached. The argument does not depend on the outcome — in the same inventory, EUR cash is 28 bp on paper and 30.43 bp delivered, dearer than 30 bp USD cash. The FX haircut reverses the ranking.

The LP satisfies coverage, eligibility, concentration, wrong-way and inventory constraints simultaneously, and returns the **shadow price of every binding constraint** — a concentration cap costing 2.61 bp is the desk's own credit policy, priced.

SETTLEMENT

Instructions are generated in ISO 20022 (`colr.003` / `colr.005`) and MT527 **message shape**, reconcile to the Delivery Amount, and are **never transmitted**. No network, no membership, no BIC. The generated XML is not schema-validated.

Part 4 — The run, end to end

1. `DOCUMENT_INGESTED` — content hash anchors the run.
2. Legal-Clause extracts; flagged fields block.
3. Reviewer confirms each, with a rationale. `TERMS_FROZEN`.
4. `run_swarm()` plans the Risk-Engine stage, validates it against the registry, executes it, and asserts exposure, initial margin and margin call are all present.
5. `collateral_optimizer_plan()` branches on the four conditions above.
6. A decision record is produced — **a proposal, pending human approval**.
7. Two runs of the same inputs produce byte-identical audit hashes. That test *is* the reproducibility claim; without it, "reproducible" is marketing.

Every API response carrying a computed number also carries `engine_version`, `snapshot_id`, `as_of` and `trace_id`.

`plan_source` records whether the routing came from the deterministic policy or from a model-produced plan. In `live` mode a model may select tools and reference arguments, but the result is validated against the registry first, and any plan naming an unknown tool, omitting a required argument or carrying a numeric argument is rejected and the deterministic policy is used instead.